

Automation with Selenium

Assignment 6: Windows, Tabs, and Iframes

1. Problem Statement: -

Open a page containing an embedded iframe and a button that opens a new tab. Use `driver.switch_to.frame()` to interact inside the iframe. Click the tab button, use `driver.window_handles` to switch context to the new tab, grab its title, close it, and switch back.

2. Objective: -

To master context switching across iframes and separate browser windows/tabs.

3. Tools, Software, and Concepts Used: -

Python 3.x, Selenium WebDriver.

4. Source Code / Implementation: -

```
from selenium import webdriver
from selenium.webdriver.common.by import By
from selenium.webdriver.support.ui import WebDriverWait
from selenium.webdriver.support import expected_conditions as EC
import time
driver = webdriver.Chrome()
driver.maximize_window()
driver.get("https://rahulshettyacademy.com/AutomationPractice/")
driver.execute_script("window.scrollTo(0, 1500);")
time.sleep(3)
driver.switch_to.frame("courses-iframe")
try:
    login_btn = WebDriverWait(driver, 10).until(
        EC.presence_of_element_located((By.CSS_SELECTOR, ".login-btn"))
    )
    print("Iframe text:", login_btn.text)
except:
    print("Iframe text: (Element not found, skipping so script doesn't crash!)")
driver.switch_to.default_content()
btn = driver.find_element(By.ID, "opentab")
driver.execute_script("arguments[0].scrollIntoView({block: 'center'});",
btn)
time.sleep(1)
btn.click()
driver.switch_to.window(driver.window_handles[1])
```

```
time.sleep(2) # Give the tab a moment to grab the title
print("New Tab Title:", driver.title)
time.sleep(10)
driver.close()
driver.switch_to.window(driver.window_handles[0])
print("Main window title:", driver.title)
driver.quit()
```

5. Output & Screenshots: -

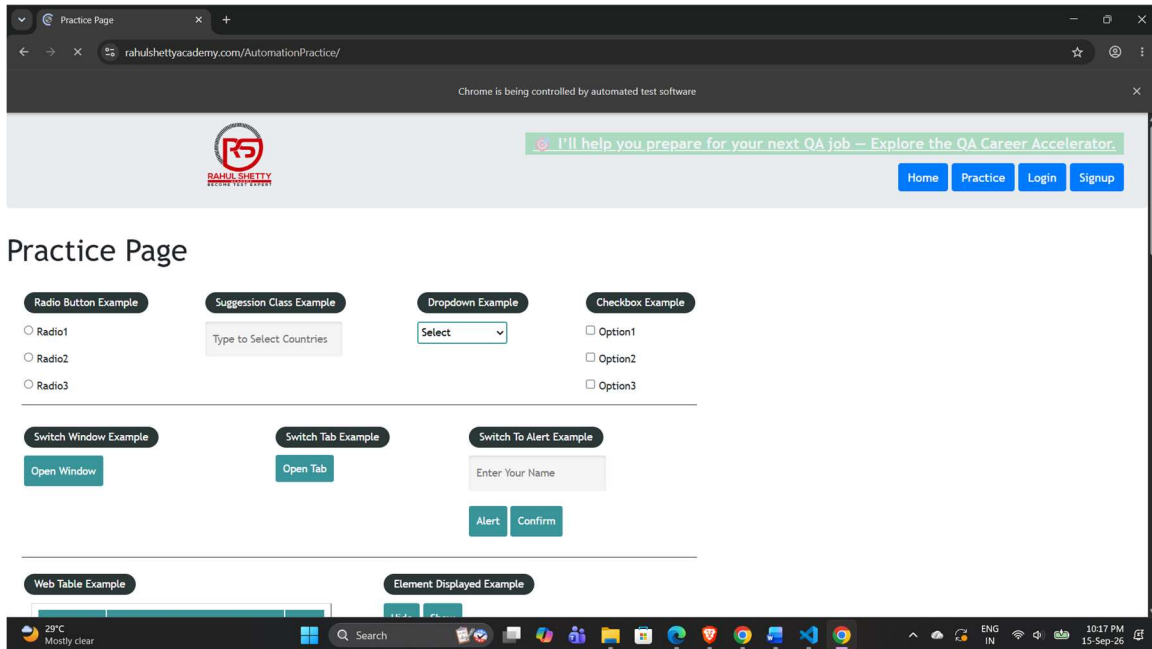


Fig.1: Main page is visible

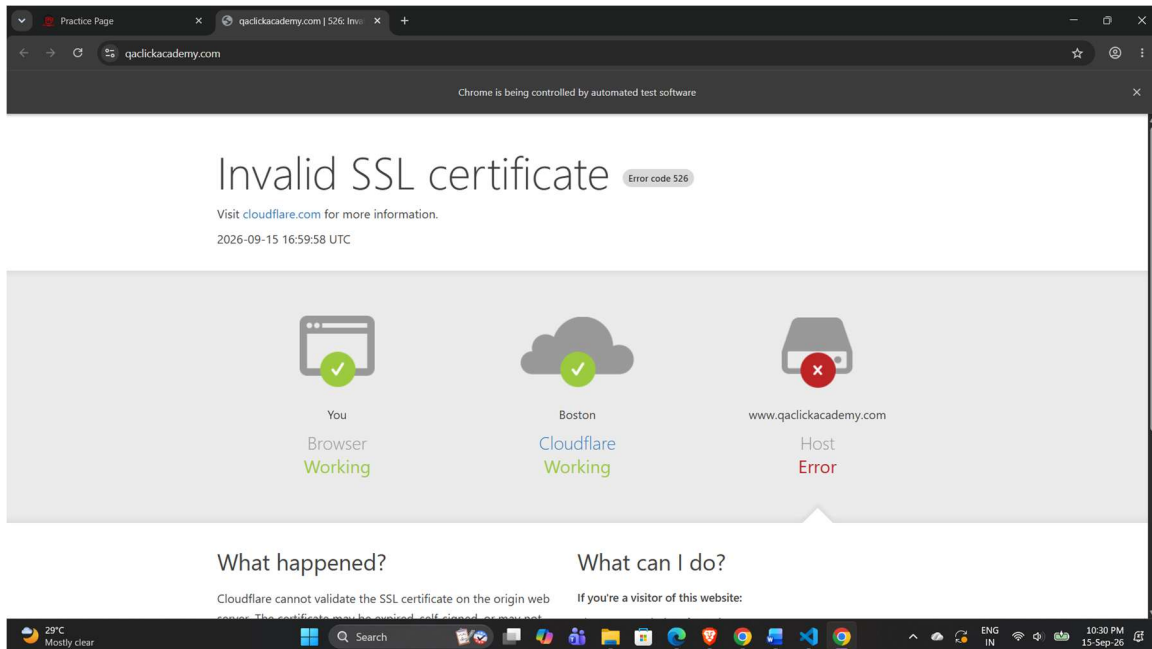


Fig.2: New tab opened

6. Result, Observation, and Conclusion: -

- Result: Successfully interacted with elements inside an iframe, spawned a new tab, and toggled control between windows.

- Observation: WebDriver can only interact with elements in the currently active context; explicit switching is necessary for nested or external frames/windows.
- Conclusion: Context management is vital for testing modern web applications that utilize pop-outs, tabs, and embedded widgets.